Video 05 — Program execution: tutorial companion

Video file: BlueImpulse_Video05_Program_Execution.mp4
Viewing position: 5 of 5
Duration: 10:00 (600 seconds)
Format: MP4, 1920 × 1080, 60 fps
File size: 45,852,242 bytes (45.85 MB)

Prerequisites: Basic arithmetic and the idea that memory locations have addresses. No assembly-language experience is assumed.

Learning goals:

- Trace fetch, decode, and execute
- Give each register its job
- Follow a four-instruction program

Chapter and subtopic connections:

- Chapter 1 — The Fundamentals / Fetch / decode / execute
- Chapter 1 — The Fundamentals / From source code to execution
- Chapter 1 — The Fundamentals / Trace a tiny program

Apply in the learning system: Complete ACT-01 in the Study Guide / Markdown activity file. Use Test Bank questions TF01–TF04, MC01–MC04 and OE01 for review. The six-chapter PPT deck introduces the same chapter concepts through classroom discussion.

Optional interactive companion: ROVERLAB lets you apply these ideas by building a robot, connecting its network and comparing stimulus-response rules. Play at https://roverlab-blue-impulse.vercel.app and use the ROVERLAB activity in the Study Guide. It is a simplified rule-and-feedback model, not a neural network or the numerical Q-learning algorithm taught in Video 01.

Materials: Paper and a calculator are sufficient. The optional supporting Python demonstration uses Python 3.9 or newer with no third-party packages or account.

```
python3 supporting/BlueImpulse_Demonstrations.py --topic execution
```

How to learn with this tutorial

Watch the opening problem, write a prediction, and follow the labelled mechanism. Pause during each learning check before reading the feedback. After the worked example, change one input and explain why the result changes. The tutorial is self-contained: all inputs, instructions and expected outputs are below.

Chapter timestamps and teaching route

Time	Teaching step	Chapter / subtopic
00:00–00:32	Where does seven go?	Chapter 1 — The Fundamentals / Fetch / decode / execute
00:32–01:11	Today's route	Chapter 1 — The Fundamentals / Fetch / decode / execute
01:11–02:00	Before the processor executes	Chapter 1 — The Fundamentals / From source code to execution
02:00–02:50	Use the recipe analogy	Chapter 1 — The Fundamentals / Fetch / decode / execute
02:50–03:54	Fetch one instruction	Chapter 1 — The Fundamentals / Fetch / decode / execute
03:54–04:41	Read the four instructions	Chapter 1 — The Fundamentals / Trace a tiny program
04:41–05:51	Single-step the worked program	Chapter 1 — The Fundamentals / Trace a tiny program
05:51–06:44	Value or address?	Chapter 1 — The Fundamentals / Trace a tiny program
06:44–07:31	The next address can change	Chapter 1 — The Fundamentals / Fetch / decode / execute
07:31–08:25	Change one instruction	Chapter 1 — The Fundamentals / Trace a tiny program
08:25–09:32	Instructions change machine state	Chapter 1 — The Fundamentals / Fetch / decode / execute
09:32–10:00	Pip completes the shift	Chapter 1 — The Fundamentals / Fetch / decode / execute

Reproduce the program trace

This invented one-word ISA is a teaching model. The # sign marks a literal value; STORE's operand is a destination address. Begin at PC 100, ACC 0 and memory[200] 0.

Fetched address	Instruction	PC after fetch	ACC after execution	memory[200]
100	LOAD #3	101	3	0
101	ADD #4	102	7	0
102	STORE 200	103	7	7
103	HALT	104	7	7

STORE copies the accumulator’s 7 to location 200; it does not store 200 or change ACC to 200. Real instruction sizes, register names and fetch timing depend on the architecture.

Practice: Replace ADD #4 with ADD #9. Predict memory[200] and explain the intermediate accumulator value.

Checked answer: LOAD makes ACC 3; ADD makes ACC 12; STORE copies 12 to memory[200]. The final PC remains 104 in this model. Changing one input and predicting every state is a stronger check than copying a highlighted answer.

Guided explanation

00:00 — Where does seven go?

Reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Three deliveries, plus four. Where does seven go? Pip's tiny demonstration program must add two values and save the result. The processor follows instruction meanings and changes stored state. It does not understand the whole program like a reader understands a story.

Have you ever wondered how a line of code becomes a visible result? Chapter one, The Fundamentals, lets us trace the registers and follow every important state change.

01:11 — Before the processor executes

Reference: Chapter 1 — The Fundamentals / From source code to execution

The subtopic From source code to execution explains that machine code contains the instructions a processor executes. An assembler translates assembly mnemonics and labels. A compiler produces another representation, often native code or bytecode. An interpreter runs operations through a runtime, sometimes working with bytecode.

The same language can have compiled and interpreted implementations, and modern systems may combine several techniques. Avoid the blanket claim that every interpreter translates exactly one source line at a time. Our demonstration uses invented assembly notation so that every state change remains easy to see.

02:00 — Use the recipe analogy

Reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Use a recipe to remember the register jobs. The program counter marks the next recipe line. The memory address register selects the location. The memory data register carries the item read from memory. The current instruction register holds the instruction being followed. The accumulator is a temporary mixing bowl for arithmetic.

We abbreviate those names as P C, M A R, M D R, C I R, and A C C. The analogy helps you remember the jobs, but these are electronic storage locations inside a processor model. Keep the jobs distinct: an address points to a location, while an instruction or value is the content being handled.

02:50 — Fetch one instruction

Reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Follow the fetch sequence. First copy the address in the program counter to the memory address register. Read the instruction at that memory location into the memory data register. Advance the program counter so it identifies the next instruction address, then copy the fetched instruction into the current instruction register.

The control unit can now decode the operation and its operand, and the processor executes the required work. In this simplified model, every instruction occupies one word, so the program counter advances by one. Real instruction sizes and exact timing depend on the architecture. Our table is a teaching model, not every processor's wiring diagram.

An address identifies a place. A value is the information stored or processed there. Keeping these roles separate makes the register trace much easier to follow.

03:54 — Read the four instructions

Reference: Chapter 1 — The Fundamentals / Trace a tiny program

In the subtopic Trace a tiny program, the first instruction sits at address one hundred: LOAD, hash three. The hash sign denotes an immediate value in this invented notation, so that instruction puts the number three into the accumulator. Address one hundred and one contains ADD, hash four. Address one hundred and two contains STORE, two hundred.

Address one hundred and three contains HALT, which ends this teaching program. The number two hundred in STORE identifies a memory location. It is not the value to put into the accumulator. Before running anything, predict what that memory location will contain when the program finishes.

04:41 — Single-step the worked program

Reference: Chapter 1 — The Fundamentals / Trace a tiny program

Let's single-step. Initially, the program counter is one hundred. Fetch the LOAD instruction and advance to one hundred and one. Decode and execute LOAD, hash three. The accumulator now contains three. Next fetch ADD, hash four from address one hundred and one. The program counter advances to one hundred and two.

Execution adds four to the accumulator's three, giving seven. Memory location two hundred has not been written yet. Fetch STORE, two hundred. The program counter advances to one hundred and three, and execution copies seven into memory location two hundred. The accumulator still contains seven. STORE copies the value; it does not turn the accumulator into an address.

Finally fetch HALT. In this simplified trace, the fetch advances the counter to one hundred and four before execution stops. The runnable Python model prints these same states and register transfers, so you can reproduce the trace yourself.

05:51 — Value or address?

Reference: Chapter 1 — The Fundamentals / Trace a tiny program

Pause and check your prediction. After the STORE instruction, what are the accumulator and memory location two hundred? Also ask yourself whether STORE, two hundred means storing the number two hundred. The accumulator remains seven, and memory location two hundred receives seven. Two hundred is the destination address.

The hash sign in our LOAD and ADD operands marks literal values. The STORE operand has a different role. This distinction prevents a common tracing mistake: confusing where data lives with the value stored there. Use this model's instruction meanings before applying assumptions from a

different programming language or processor.

Check your answer: ACC = 7; memory[200] = 7

06:44 — The next address can change

Reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

A branch instruction may replace the normal next value of the program counter. That is how the instruction sequence can follow a decision or repeat a loop instead of always moving forward. Our four-instruction program contains no branch, so the normal next addresses stay easy to follow.

Chapter three, Hardware, adds pipelining: real processors may overlap instructions while preserving the required program behaviour. Overlapping the implementation does not make the logical effects of a program arbitrary. For a basic trace, first understand the defined instruction effects, then study how a specific processor implements them.

07:31 — Change one instruction

Reference: Chapter 1 — The Fundamentals / Trace a tiny program

Your turn. Change ADD, hash four to ADD, hash nine, while keeping LOAD, hash three and STORE, two hundred. What value should reach memory location two hundred? Work through the accumulator before we reveal it. LOAD gives three. Adding nine gives twelve. STORE copies twelve into memory location two hundred.

That is a small example of testing a prediction by changing one input to the program. The included simulator accepts a different add value, prints the trace, and checks that the final memory result matches the arithmetic. Try another value and explain each register's role aloud as the instruction moves through the cycle.

Check your answer: memory[200] = 3 + 9 = 12

08:25 — Instructions change machine state

Reference: Chapter 1 — The Fundamentals / Fetch / decode / execute

Recap the lesson. Source code reaches execution through a translator or runtime appropriate to its implementation. The program counter identifies the next instruction. Fetch brings it from memory, decode interprets it, and execute performs its operation. The memory address register selects a location, the memory data register carries data, and the current instruction register holds the operation.

Our teaching accumulator went from three to seven, and STORE copied seven to memory location two hundred. Addresses and values have different roles. Branches can change the normal next instruction address. Complete the trace without watching the highlight. Understanding means predicting each state change yourself.

Pip's seven is now a stored result because STORE copied the accumulator value to the chosen memory address. The instruction determined both what changed and what stayed the same.

Finish by explaining, not copying

Complete ACT-01 and the named Test Bank questions, then explain the mechanism without following the highlight. State one assumption of the worked model and one condition that could make the outcome differ in a real system. A correct calculation supports only the conclusion it actually measures.